

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 6 — Solutions

Due to the Easter break, you have two weeks to work on this exercise sheet and it covers 1.5 times the usual content (i.e., 6 slide sets and exercises worth 15 marks).

Exercise 6.1 (1+0.5+0.5 marks)

A *Hamiltonian path* of a directed graph $G = \langle V, E \rangle$ is a path through G that visits each vertex $v \in V$ exactly once. The *maximum Hamiltonian path* of a directed graph G with (positive) weighted edges is the Hamiltonian path with the maximal sum of edge weights.

- (a) Formalize the combinatorial optimization problem (COP) of finding a maximum Hamiltonian path in a fully connected graph, i.e., there is an edge between all pairs of vertices in the graph.
- (b) Is your formulation a pure search problem, a pure optimization problem, or a combined search and optimization problem?
- (c) Define a suitable neighborhood for hill climbing by providing a formal definition of a function mapping a candidate to a set of neighbors. A neighborhood function is suitable for hill climbing if it ensures that every candidate can be reached and if it bounds the amount of neighbors to a reasonable size. You do not need to show that the function is suitable.

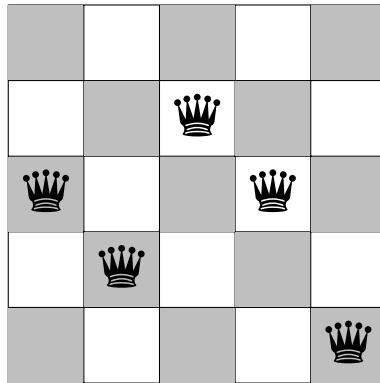
Solution:

- (a) For a given directed graph $G = \langle V, E \rangle$ with edge-weight function w , the maximum Hamiltonian path problem is the combinatorial optimization problem $\langle C, S, \text{opt}, v \rangle$ with
 - C the set of all permutations of V , i.e., $|V|$ -tuples $\langle v_1, \dots, v_{|V|} \rangle$ such that $i \neq j$ implies $v_i \neq v_j$ for all $1 \leq i, j \leq |V|$
 - $S = C$
 - $\text{opt} = \max$
 - $v(\langle v_1, \dots, v_{|V|} \rangle) = \sum_{i=1}^{|V|-1} w((v_i, v_{i+1}))$
- (b) Solutions of different quality exist and all candidates are solutions. Therefore, this is a pure optimization problem.
- (c) Let $n = |V|$. Given a candidate $\langle v_1, \dots, v_n \rangle \in C$, we define the neighboring function $n(\langle v_1, \dots, v_i, v_{i+1}, v_n \rangle) = \{\langle v_1, \dots, v_{i+1}, v_i, \dots, v_n \rangle \mid 1 \leq i \leq n\}$. In other words, given a permutation of V , a neighbor swaps two consecutive vertices in the represented Hamiltonian path.

With this neighboring function we can reach any permutation $\langle v_1, \dots, v_n \rangle$ from any other permutation $\langle v'_1, \dots, v'_n \rangle$; for example, if $v_n = v'_i$, then we can “bubble” v'_i to the last position of the tuple by consecutively swapping v'_i with v'_{i+1} until $i = n$. Then we can continue with v_{n-1} and so on, until $v_i = v'_i$ for all $1 \leq i \leq n$. Furthermore, each candidate has only $|V| - 1$ neighbors.

Exercise 6.2 (1.5+1.5 marks)

The 5-queens problem is the COP on a 5×5 chess board that is analogous to the 8-queens problem that has been introduced in Chapter C1 in the lecture. In this exercise, we are interested in finding a solution for the 5-queens problem with different local search variants. Consider the following initial candidate:



- (a) Depict the candidates that are considered by hill climbing with the heuristic and the neighborhood that are introduced on slide 23 of Chapter C1 in the print version of the lecture slides. If necessary, break ties in favor of candidates where the leftmost possible queen is moved to a square that is as far up as possible. Annotate the candidates that are considered during search with the heuristic values of all neighbors.

Solution:

The numbers encode the number of queen pairs with conflict in the initial candidate:

2	4	3	2	4
2	5		3	5
	5	6		5
3		4	3	6
3	4	5	2	

As the heuristic value of the initial candidate is 3, a candidate with heuristic value of 2 is selected. With the given tie breaking rule, the next candidate (and the heuristic values of that candidates neighbors) is the following:

	5	2	3	3
2	5		3	3
3	4	5		2
3		3	4	4
3	4	3	2	

Hill climbing terminates after this step (without finding a solution) as there is no candidate with lower heuristic value.

- (b) Now consider a hill climbing variant where the next candidate that is considered is selected by first picking a file at random and then moving the (unique) queen in that file to a square in that file with the best heuristic value (the variant is introduced on slide 9 of Chapter C2 in the print version of the lecture slides). Assume that the random choices are resolved in such a way that the file selection picks the third file first, and in each subsequent step the file that is to the left of the file that was selected last (if the leftmost file was selected last, the rightmost file is selected). Provide the candidates that are considered if the heuristic is used that counts the number of queens that threaten the queen in the selected file. If necessary, break ties in favor of candidates where the queen that is moved is as far up as possible. Annotate the candidates that are considered during search with the heuristic values of all neighbors.

Solution:

The heuristic values of the initial candidate are:

		1		
				
		4		
		2		
		3		

As the heuristic value of the candidate that is created with a null move is 1 as well, the following candidate is considered next due to the tie breaking criteria:

	2			
	3			
	2			
				
	2			

As the heuristic value of the null move candidate is 1, no queen is moved.

2		♔		
0				
♔			♔	
1	♔			
2				♔

After moving the queen to the candidate with a heuristic value of 0, the algorithm terminates with the following solution:

		♔		
♔				
			♔	
	♔			
				♔

Note that the file chess-board.tex, which you can find on the ADAM website, contains the LaTeX source code for the 5×5 chess board depicted above.

Exercise 6.3 (2+0.5+0.5 marks)

Consider the propositional formula $\varphi = (a \vee b) \wedge (\neg b \vee c) \wedge (\neg a \vee \neg b)$.

- Formalize φ as a binary constraint network.
- Provide a consistent partial variable assignment.
- Provide an inconsistent partial variable assignment.

Solution:

- $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ where

- $V = \{a, b, c\}$
- $\text{dom}(x) = \{\mathbf{T}, \mathbf{F}\}$ for all $x \in V$
- $R_{ab} = \{\langle \mathbf{T}, \mathbf{F} \rangle, \langle \mathbf{F}, \mathbf{T} \rangle\}$
 $R_{ac} = \{\langle \mathbf{T}, \mathbf{T} \rangle, \langle \mathbf{T}, \mathbf{F} \rangle, \langle \mathbf{F}, \mathbf{T} \rangle, \langle \mathbf{F}, \mathbf{F} \rangle\}$
 $R_{bc} = \{\langle \mathbf{T}, \mathbf{T} \rangle, \langle \mathbf{F}, \mathbf{T} \rangle, \langle \mathbf{F}, \mathbf{F} \rangle\}$

- for example $\alpha_1 = \{a \mapsto \mathbf{T}, b \mapsto \mathbf{F}\}$ or $\alpha_2 = \{\}$
- for example $\alpha = \{b \mapsto \mathbf{T}, c \mapsto \mathbf{F}\}$

Exercise 6.4 (1.5+0.5+0.5+1.5 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with $V = \{w, x, y, z\}$ and

$$\begin{array}{ll} \text{dom}(w) = \{1, 2, 3, 4\} & R_{wy} = \{\langle 3, 3 \rangle\} \\ \text{dom}(x) = \{1, 2, 3\} & R_{wz} = \{\langle 1, 4 \rangle, \langle 2, 4 \rangle, \langle 3, 2 \rangle, \langle 4, 4 \rangle\} \\ \text{dom}(y) = \{1, 3, 4\} & R_{xz} = \{\langle 1, 4 \rangle, \langle 2, 2 \rangle\} \\ \text{dom}(z) = \{2, 4\} & R_{yz} = \{\langle 3, 2 \rangle, \langle 3, 4 \rangle\} \end{array}$$

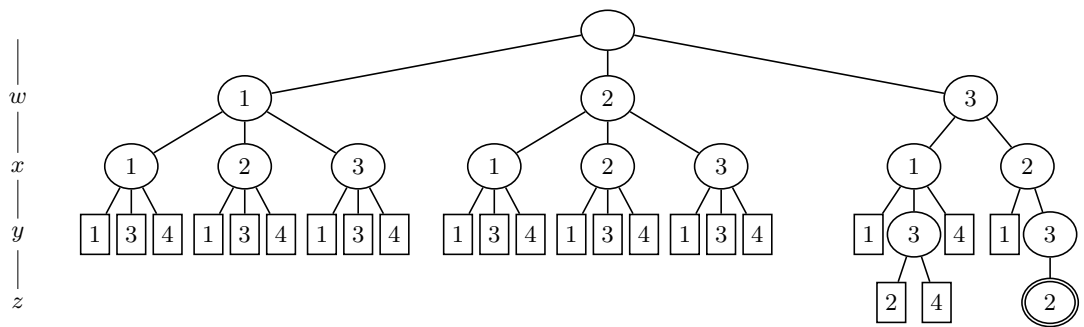
- (a) Provide the search tree that is created by applying naive backtracking on $\mathcal{C}$ as introduced in chapter D3. Choose variables in alphabetical order and values in ascending order. Depict the search tree in the same style as the lecture slides, with nodes ordered from left to right according to the order in which they are explored by the algorithm.
- (b) Consider the following static order strategy for the variables:
- Select a variable according to the *minimum remaining values* variable order criterion.
 - If there is more than one such variable, break ties according to the *most constraining variable* variable order criterion.
 - If the choice is still not unique, break ties alphabetically.

Provide the resulting variable order.

- (c) For each variable in the order of part (b), order its values according to the *minimum conflicts* criterion (number of conflicts with *unassigned* variables, i.e., variables later in the variable order). Break ties in favor of lower values as in part (a).
- (d) Now provide the search tree that is created by applying naive backtracking using the variable order from (b) and the value order from part (c). Compare the result to the result of (a).

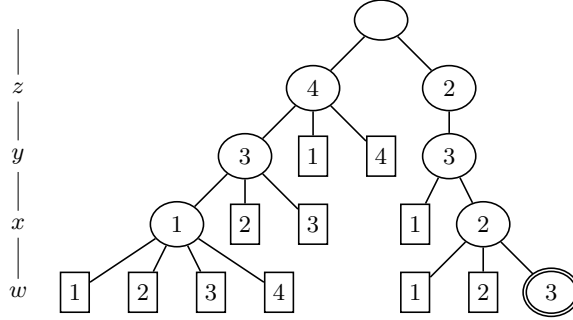
Solution:

- (a) Naive backtracking with alphabetical variable order and ascending value order results in the following search tree:



- (b) The resulting variable order is $z \prec y \prec x \prec w$.
- (c)
- The value order for z is $4 \prec 2$.
($z \mapsto 4$ has $2 + 2 + 1$ conflicts with y, x, w respectively, while value $z \mapsto 2$ has $2 + 2 + 3$.)
 - The value order for y is $3 \prec 1 \prec 4$.
($y \mapsto 3$ has $0 + 3$ conflicts with x, w respectively, while $y \mapsto 1$ and $y \mapsto 4$ have $0 + 4$.)
 - The value order for x is $1 \prec 2 \prec 3$.
(None of the assignments of x have conflicts with w .)

- The value order for w is $1 \prec 2 \prec 3 \prec 4$.
(There are no unassigned variables at this point in the variable order.)
- (d) Naive backtracking with the above variable and value order results in the following search tree:



The orders defined in (b) and (c) were better: the search tree in (d) has only 18 nodes while the one in (a) has 37.

Exercise 6.5 (1+2 marks)

Consider the constraint network $\mathcal{C} = \langle V, \text{dom}, (R_{uv}) \rangle$ with

- $V = \{a, b, c\}$
- $\text{dom}(x) = \{0, 1, 2, 3\}$ for all $x \in V$
- $R_{a,b} = \{\langle 0, 0 \rangle, \langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 3 \rangle\}$
 $R_{a,c} = \{\langle 0, 3 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle, \langle 3, 0 \rangle\}$
 $R_{b,c} = \{\langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 0, 3 \rangle, \langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 2, 3 \rangle\}$

- (a) Consider the partial assignment $\alpha = \{b \mapsto 0\}$. Specify the domains of all variables after applying forward checking in α .
- (b) Define the tightest network $\mathcal{C}'$ that is equivalent to $\mathcal{C}$. In other words, find the unique $\mathcal{C}'$ such that $\mathcal{C}' \equiv \mathcal{C}$ and there is no $\mathcal{C}''$ with $\mathcal{C}'' \sqsubset \mathcal{C}'$ and $\mathcal{C}'' \equiv \mathcal{C}$. Justify your solution.

Solution:

- (a) $\text{dom}(a) = \{0\}$, $\text{dom}(b) = \{0\}$, $\text{dom}(c) = \{1, 2, 3\}$
- (b) The constraints state that $a = b$, $a + c = 3$ and $b < c$ must hold. Since we have $a = b$ and $b < c$ we can infer $a < c$, which means there are only two ways to achieve $a + c = 3$: with $a = 0$ and $c = 3$, or $a = 1$ and $c = 2$. There are thus only two solutions to the constraint network: $S_1 = \{a \mapsto 0, b \mapsto 0, c \mapsto 3\}$ and $S_2 = \{a \mapsto 1, b \mapsto 1, c \mapsto 2\}$.

The tightest network with exactly solutions S_1 and S_2 must reduce the domains and constraints of the variables to only contain values and relations that occur in some solution.

Concretely we obtain $\mathcal{C}' = \langle V', \text{dom}', (R'_{uv}) \rangle$ with

- $V' = V$
- $\text{dom}'(a) = \text{dom}'(b) = \{0, 1\}$ and $\text{dom}'(c) = \{2, 3\}$
- $R'_{a,b} = \{\langle 0, 0 \rangle, \langle 1, 1 \rangle\}$
 $R'_{a,c} = \{\langle 0, 3 \rangle, \langle 1, 2 \rangle\}$
 $R'_{b,c} = \{\langle 0, 3 \rangle, \langle 1, 2 \rangle\}$